

PROJET 9

October 7, 2025

```
[1]: import pandas as pd

[2]: import plotly.express as px

[3]: pd.set_option('display.max_columns', None)

[4]: import warnings
warnings.simplefilter("ignore")

[5]: cust = pd.read_csv("/Users/lyrab/Documents/PROJETS/PROJET9/customers.csv",
    ↪sep=";")

prod = pd.read_csv("/Users/lyrab/Documents/PROJETS/PROJET9/products.csv", sep=";
    ↪")

tran = pd.read_csv(
    "/Users/lyrab/Documents/PROJETS/PROJET9/Transactions.csv",
    sep=";",
    dtype={"id_prod": str, "session_id": str, "client_id": str},
    parse_dates=["date"]
)
```

1 Analyse Customers

```
[6]: #Dimensions du dataset :
print("Le tableau comporte {} observation(s) ou client(s)".format(cust.
    ↪shape[0]))
print("Le tableau comporte {} colonne(s)".format(cust.shape[1]))
```

Le tableau comporte 8621 observation(s) ou client(s)
Le tableau comporte 3 colonne(s)

```
[7]: # nombre de colonnes
print(f"Nombre de colonnes : {cust.shape[1]}")

# nature des données pour chaque colonne
print(cust.dtypes)
```

```
# le nombre de valeurs non nulles par colonne
print(cust.count())

# un résumé complet du DataFrame
print(cust.info())
```

```
Nombre de colonnes : 3
client_id    object
sex          object
birth        int64
dtype: object
client_id    8621
sex          8621
birth        8621
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8621 entries, 0 to 8620
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   client_id    8621 non-null    object
1   sex          8621 non-null    object
2   birth        8621 non-null    int64
dtypes: int64(1), object(2)
memory usage: 202.2+ KB
None
```

```
[8]: cust.head()
```

```
[8]:  client_id sex  birth
0    c_4410  f   1967
1    c_7839  f   1975
2    c_1699  f   1984
3    c_5961  f   1962
4    c_5320  m   1943
```

```
[9]: # Doublons complets (lignes identiques)
print("Nombre de doublons complets :", cust.duplicated().sum())

# Doublons sur la clé primaire client_id
print("client_id est une clé primaire :", cust["client_id"].nunique() == cust.
      ↪shape[0])
```

```
Nombre de doublons complets : 0
client_id est une clé primaire : True
```

```
[10]: # Afficher les valeurs distinctes d'une colonne catégorielle
print(cust['sex'].unique())
```

```
# Compter le nombre d'occurrences pour chaque modalité
print(cust['sex'].value_counts())
```

```
['f' 'm']
sex
f    4490
m    4131
Name: count, dtype: int64
```

```
[11]: # Compter les valeurs manquantes par colonne
```

```
print(cust.isnull().sum())
```

```
# Afficher la proportion de valeurs manquantes
```

```
print(cust.isnull().mean().round(4) * 100)
```

```
client_id    0
sex          0
birth        0
dtype: int64
client_id    0.0
sex          0.0
birth        0.0
dtype: float64
```

```
[12]: # Valeurs distinctes dans la colonne 'sex'
```

```
print("Valeurs distinctes dans 'sex' :", cust['sex'].unique())
```

```
# Statistiques sur 'birth'
```

```
print("Statistiques sur birth :", cust['birth'].describe())
```

```
# Calcul de l'âge pour détecter les valeurs extrêmes
```

```
cust['age'] = 2025 - cust['birth']
```

```
print("Âges minimum et maximum :", cust['age'].min(), cust['age'].max())
```

```
Valeurs distinctes dans 'sex' : ['f' 'm']
Statistiques sur birth : count    8621.000000
mean    1978.275606
std      16.917958
min     1929.000000
25%     1966.000000
50%     1979.000000
75%     1992.000000
max     2004.000000
Name: birth, dtype: float64
Âges minimum et maximum : 21 96
```

2 Analyse de la table products

```
[13]: # Dimensions du tableau
print(f"Le tableau comporte {prod.shape[0]} observation(s) ou article(s)")
print(f"Le tableau comporte {prod.shape[1]} colonne(s)")
```

Le tableau comporte 3286 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)

```
[14]: # Types de données
print(prod.dtypes)
```

```
id_prod    object
price      float64
categ      int64
dtype: object
```

```
[15]: # Affichage des premières lignes
print(prod.head())
```

```
   id_prod  price  categ
0  0_1421   19.99      0
1  0_1368    5.13      0
2   0_731   17.99      0
3   1_587    4.99      1
4  0_1507    3.99      0
```

```
[16]: # Vérification des valeurs manquantes
print(prod.isnull().sum())
```

```
id_prod    0
price      0
categ      0
dtype: int64
```

```
[17]: # Proportion de valeurs manquantes
print((prod.isnull().mean() * 100).round(2))
```

```
id_prod    0.0
price      0.0
categ      0.0
dtype: float64
```

```
[18]: # Vérification des doublons complets
print("Nombre de doublons complets :", prod.duplicated().sum())
```

Nombre de doublons complets : 0

```
[19]: # Vérification unicité de la colonne id_prod
```

```
print("id_prod est une clé primaire :", prod["id_prod"].nunique() == prod.  
      ↪shape[0])
```

id_prod est une clé primaire : True

```
[20]: # Valeurs distinctes dans la colonne 'categ'  
print("Valeurs distinctes dans 'categ' :", prod['categ'].unique())
```

Valeurs distinctes dans 'categ' : [0 1 2]

```
[21]: # Statistiques sur la colonne price  
print("Statistiques sur 'price' :")  
print(prod['price'].describe())
```

Statistiques sur 'price' :

count	3286.000000
mean	21.863597
std	29.849786
min	0.620000
25%	6.990000
50%	13.075000
75%	22.990000
max	300.000000

Name: price, dtype: float64

```
[22]: # Vérification des prix : y a-t-il des prix non renseignés, négatifs ou nuls ?  
  
# Afficher le nombre d'articles avec un prix non renseigné  
print("Nombre d'article(s) avec un prix non renseigné :", prod['price'].isna().  
      ↪sum())  
  
# Afficher le prix minimum  
print("Prix minimum :", prod['price'].min())  
  
# Afficher le prix maximum  
print("Prix maximum :", prod['price'].max())  
  
# Afficher les lignes avec un prix inférieur à 0 (à analyser manuellement ↵  
      ↪ensuite)  
print("Articles avec un prix négatif :")  
print(prod[prod['price'] < 0])
```

Nombre d'article(s) avec un prix non renseigné : 0
Prix minimum : 0.62
Prix maximum : 300.0
Articles avec un prix négatif :
Empty DataFrame
Columns: [id_prod, price, categ]
Index: []

```
[23]: # Valeurs uniques dans la colonne 'categ'
print("Valeurs distinctes dans 'categ' :", prod['categ'].unique())
```

Valeurs distinctes dans 'categ' : [0 1 2]

```
[24]: # Nombre d'occurrences par catégorie
print(prod['categ'].value_counts())
```

```
categ
0    2308
1     739
2     239
Name: count, dtype: int64
```

```
[25]: # Type de la colonne (doit être int ou object selon la sémantique)
print(prod['categ'].dtype)
```

int64

```
[26]: # Valeurs manquantes dans la colonne 'categ'
print("Valeurs manquantes dans 'categ' :", prod['categ'].isnull().sum())
```

Valeurs manquantes dans 'categ' : 0

```
[27]: # Afficher les lignes avec des catégories inhabituelles (ex : hors 0, 1, 2)
print("Valeurs inattendues dans 'categ' :")
print(prod[~prod['categ'].isin([0, 1, 2])])
print(" Toutes les valeurs de la colonne 'categ' sont valides et comprises_
↳ dans les modalités attendues (0, 1, 2). Aucun nettoyage nécessaire à cette_
↳ étape.")
```

Valeurs inattendues dans 'categ' :
Empty DataFrame
Columns: [id_prod, price, categ]
Index: []

Toutes les valeurs de la colonne 'categ' sont valides et comprises dans les modalités attendues (0, 1, 2). Aucun nettoyage nécessaire à cette étape.

3 Analyse de la Table Transactions

```
[28]: # Dimensions du tableau
print(f"Le tableau comporte {tran.shape[0]} ligne(s) de transaction")
print(f"Le tableau comporte {tran.shape[1]} colonne(s)")
```

Le tableau comporte 1048575 ligne(s) de transaction
Le tableau comporte 4 colonne(s)

```
[29]: # Types de données
print(tran.dtypes)
```

```

id_prod      object
date         datetime64[ns]
session_id   object
client_id    object
dtype: object

```

```

[30]: # Affichage des premières lignes
print(tran.head())

```

```

   id_prod      date session_id client_id
0  0_1259 2021-03-01 00:01:07.843138      s_1      c_329
1  0_1390 2021-03-01 00:02:26.047414      s_2      c_664
2  0_1352 2021-03-01 00:02:38.311413      s_3      c_580
3  0_1458 2021-03-01 00:04:54.559692      s_4     c_7912
4  0_1358 2021-03-01 00:05:18.801198      s_5     c_2033

```

```

[31]: # Vérification des valeurs manquantes
print("Valeurs manquantes par colonne :")
print(tran.isnull().sum())

```

```

Valeurs manquantes par colonne :
id_prod      361041
date         361041
session_id   361041
client_id    361041
dtype: int64

```

```

[32]: # Proportion de valeurs manquantes
print("Pourcentage de valeurs manquantes :")
print((tran.isnull().mean() * 100).round(2))

```

```

Pourcentage de valeurs manquantes :
id_prod      34.43
date         34.43
session_id   34.43
client_id    34.43
dtype: float64

```

```

[33]: # Vérification des doublons complets
print("Nombre de doublons complets :", tran.duplicated().sum())

```

```

Nombre de doublons complets : 361040

```

```

[34]: # Vérification de l'unicité des identifiants (session_id + id_prod ?)
print("Nombre de transactions uniques (session_id + id_prod) :",
      tran[['session_id', 'id_prod']].drop_duplicates().shape[0])

```

```

Nombre de transactions uniques (session_id + id_prod) : 686680

```

```
[35]: # Format de la date : on vérifie si c'est bien de type datetime
print("Type de la colonne 'date' :", tran['date'].dtype)
```

Type de la colonne 'date' : datetime64[ns]

```
[36]: # Création d'une version nettoyée de tran, sans les lignes contenant des NaN
tran_cleaned = tran.dropna()

# Affichage du nombre de lignes conservées
print(f"Nombre de lignes conservées après suppression des NaN : {tran_cleaned.
↳shape[0]}")
```

Nombre de lignes conservées après suppression des NaN : 687534

```
[37]: # Combien de doublons sur les couples session_id + id_prod ?
duplicated_pairs = tran_cleaned.duplicated(subset=['session_id', 'id_prod']).
↳sum()
print(f"Nombre de doublons sur session_id + id_prod : {duplicated_pairs}")
```

Nombre de doublons sur session_id + id_prod : 855

```
[38]: # Supprimer les doublons sur le couple session_id + id_prod
tran_cleaned = tran_cleaned.drop_duplicates(subset=['session_id', 'id_prod'])
```

```
[39]: # Combien de doublons sur les couples session_id + id_prod ?
duplicated_pairs = tran_cleaned.duplicated(subset=['session_id', 'id_prod']).
↳sum()
print(f"Nombre de doublons sur session_id + id_prod : {duplicated_pairs}")
```

Nombre de doublons sur session_id + id_prod : 0

4 JOINTURES

```
[40]: idprod_in_prod = tran_cleaned['id_prod'].isin(prod['id_prod']).all()
print("Tous les id_prod de tran_cleaned sont présents dans prod :",
↳idprod_in_prod)
```

Tous les id_prod de tran_cleaned sont présents dans prod : True

```
[41]: clientid_in_cust = tran_cleaned['client_id'].isin(cust['client_id']).all()
print("Tous les client_id de tran_cleaned sont présents dans cust :",
↳clientid_in_cust)
```

Tous les client_id de tran_cleaned sont présents dans cust : True

```
[42]: print("prod[id_prod] est unique :", prod['id_prod'].is_unique)
print("cust[client_id] est unique :", cust['client_id'].is_unique)
```

prod[id_prod] est unique : True
cust[client_id] est unique : True


```
[43]: # Jointure transactions + produits
tran_prod = tran_cleaned.merge(prod, on='id_prod', how='inner')
```

```
[44]: # Dimensions avant jointure
print("Taille de tran_cleaned :", tran_cleaned.shape)
print("Taille de prod :", prod.shape)

# Jointure sur id_prod
tran_prod = tran_cleaned.merge(prod, on='id_prod', how='inner')

# Taille après jointure
print("Taille après jointure (tran_prod) :", tran_prod.shape)
```

Taille de tran_cleaned : (686679, 4)
Taille de prod : (3286, 3)
Taille après jointure (tran_prod) : (686679, 6)

```
[45]: # Jointure du résultat précédent avec les clients
df_final = tran_prod.merge(cust, on='client_id', how='inner')
```

```
[46]: # Dimensions avant jointure
print("Taille de tran_prod :", tran_prod.shape)
print("Taille de cust :", cust.shape)

# Jointure sur client_id
df_final = tran_prod.merge(cust, on='client_id', how='inner')

# Taille après jointure finale
print("Taille après jointure (df_final) :", df_final.shape)
```

Taille de tran_prod : (686679, 6)
Taille de cust : (8621, 4)
Taille après jointure (df_final) : (686679, 9)

5 Analyses

```
[47]: # Création de la colonne CA (chiffre d'affaires)
df_final['CA'] = df_final['price']
```

```
[48]: # 1) Date
df_final['date'] = pd.to_datetime(df_final['date'], errors='coerce').dt.
    floor('D')

# 2) CA quotidien (en comblant les jours manquants à 0)
daily_ca = (
    df_final.groupby('date', as_index=False)['CA'].sum()
    .set_index('date')
```

```

        .asfreq('D', fill_value=0)
        .reset_index()
    )

    # 3) Moyenne mobile paramétrable en jours
    window_days = 7 # ex : 7 jours, mettre 30 pour 30 jours, etc.
    daily_ca['moyenne_mobile'] = daily_ca['CA'].rolling(window=window_days,
        ↪min_periods=1).mean()

    # 4) Graphique (Plotly)
    fig = px.line(
        daily_ca,
        x='date',
        y=['CA', 'moyenne_mobile'],
        labels={'date': 'Date', 'value': "Chiffre d'affaires (€)", 'variable':
        ↪'Ligne'},
        title=f"Évolution quotidienne du chiffre d'affaires avec moyenne mobile
        ↪({window_days} jours)"
    )
    fig.update_layout(template='plotly_white', xaxis_title="Jour",
        ↪yaxis_title="Chiffre d'affaires (€)", hovermode='x unified',
        ↪legend_title="Ligne")
    fig.show()

```

```

[49]: # Regrouper le chiffre d'affaires par catégorie
ca_par_categ = df_final.groupby('categ', as_index=False)['CA'].sum()

# Tracer le graphique interactif
fig = px.bar(
    ca_par_categ,
    x='categ',
    y='CA',
    title='Chiffre d'affaires par catégorie de produit',
    labels={'categ': 'Catégorie', 'CA': 'Chiffre d'affaires (€)'},
    text='CA'
)

fig.update_traces(texttemplate='%{text:.0f} €', textposition='outside')
fig.update_layout(
    yaxis_title="Chiffre d'affaires (€)",
    xaxis_title="Catégorie",
    uniformtext_minsize=8,
    uniformtext_mode='hide',
    template="plotly_white"
)

fig.show()

```

```
[50]: df_final['month'] = df_final['date'].dt.to_period('M').dt.to_timestamp()
clients_par_mois = df_final.groupby('month')['client_id'].nunique().
    ↪reset_index()
clients_par_mois.columns = ['month', 'nb_clients']

fig = px.line(
    clients_par_mois,
    x='month',
    y='nb_clients',
    title="Nombre de clients uniques par mois",
    labels={'month': 'Mois', 'nb_clients': 'Nombre de clients'},
    markers=True
)

fig.update_layout(
    template="plotly_white",
    xaxis_title="Mois",
    yaxis_title="Nombre de clients uniques",
    hovermode="x unified"
)

fig.show()
```

```
[51]: df_final['month'] = df_final['date'].dt.to_period('M').dt.to_timestamp()
transactions_par_mois = df_final.groupby('month')['session_id'].count().
    ↪reset_index()
transactions_par_mois.columns = ['month', 'nb_transactions']

fig = px.line(
    transactions_par_mois,
    x='month',
    y='nb_transactions',
    title="Nombre de transactions par mois",
    labels={'month': 'Mois', 'nb_transactions': 'Nombre de transactions'},
    markers=True
)

fig.update_layout(
    template="plotly_white",
    xaxis_title="Mois",
    yaxis_title="Nombre de transactions",
    hovermode="x unified"
)

fig.show()
```

```
[52]: df_final['month'] = df_final['date'].dt.to_period('M').dt.to_timestamp()
produits_par_mois = df_final.groupby('month')['id_prod'].count().reset_index()
produits_par_mois.columns = ['month', 'nb_produits_vendus']

fig = px.line(
    produits_par_mois,
    x='month',
    y='nb_produits_vendus',
    title="Nombre de produits vendus par mois",
    labels={'month': 'Mois', 'nb_produits_vendus': 'Produits vendus'},
    markers=True
)

fig.update_layout(
    template="plotly_white",
    xaxis_title="Mois",
    yaxis_title="Nombre de produits vendus",
    hovermode="x unified"
)

fig.show()
```

```
[53]: # Top 10 références produits par chiffre d'affaires
top_refs = (
    df_final.groupby('id_prod')['CA']
        .sum()
        .nlargest(10)
        .reset_index()
)

fig = px.bar(
    top_refs,
    x='id_prod',
    y='CA',
    title='Top 10 références par chiffre d'affaires',
    labels={'id_prod': 'Référence produit', 'CA': 'Chiffre d'affaires (€)'},
    text='CA',
    width=900,    # Largeur en pixels
    height=500    # Hauteur en pixels
)

fig.update_traces(texttemplate='%{text:.0f} €', textposition='outside')
fig.update_layout(
    template='plotly_white',
    xaxis_title='Référence produit',
    yaxis_title='Chiffre d'affaires (€)',
    uniformtext_minsize=8,
)
```

```

        uniformtext_mode='hide'
    )

fig.show()

```

```

[54]: # Flop 10 références produits par chiffre d'affaires (hors CA = 0 si besoin)
flop_refs = (
    df_final.groupby('id_prod')['CA']
        .sum()
        .nsmallest(10)
        .reset_index()
)

fig = px.bar(
    flop_refs,
    x='id_prod',
    y='CA',
    title='Flop 10 références par chiffre d'affaires',
    labels={'id_prod': 'Référence produit', 'CA': 'Chiffre d'affaires (€)'},
    text='CA',
    width=900,
    height=500
)

fig.update_traces(texttemplate='%{text:.0f} €', textposition='outside')
fig.update_layout(
    template='plotly_white',
    xaxis_title='Référence produit',
    yaxis_title='Chiffre d'affaires (€)',
    uniformtext_minsize=8,
    uniformtext_mode='hide'
)

fig.show()

```

```

[55]: # Répartition du chiffre d'affaires par catégorie
ca_par_categ = df_final.groupby('categ', as_index=False)['CA'].sum()

fig = px.pie(
    ca_par_categ,
    names='categ',
    values='CA',
    title="Répartition du chiffre d'affaires par catégorie",
    hole=0.4, # Pour un style donut
    width=800,
    height=500
)

```

```
fig.update_traces(textposition='inside', textinfo='percent+label')
fig.update_layout(template='plotly_white')

fig.show()
```

```
[56]: # Nombre de clients distincts par sexe
clients_par_sexe = df_final.groupby('sex')['client_id'].nunique().reset_index()
clients_par_sexe.columns = ['sex', 'nb_clients']

fig = px.bar(
    clients_par_sexe,
    x='sex',
    y='nb_clients',
    title='Répartition du nombre de clients par sexe',
    labels={'sex': 'Sexe', 'nb_clients': 'Nombre de clients'},
    text='nb_clients',
    width=700,
    height=500
)

fig.update_traces(textposition='outside')
fig.update_layout(
    template='plotly_white',
    xaxis_title='Sexe',
    yaxis_title='Nombre de clients'
)

fig.show()
```

```
[57]: # Chiffre d'affaires cumulé par sexe
ca_par_sexe = df_final.groupby('sex')['CA'].sum().reset_index()

fig = px.bar(
    ca_par_sexe,
    x='sex',
    y='CA',
    title="Répartition du chiffre d'affaires par sexe",
    labels={'sex': 'Sexe', 'CA': 'Chiffre d'affaires (€)'},
    text='CA',
    width=700,
    height=500
)

fig.update_traces(texttemplate='%{text:.0f} €', textposition='outside')
fig.update_layout(
    template='plotly_white',
```

```

    xaxis_title='Sexe',
    yaxis_title='Chiffre d'affaires (€)'
)

fig.show()

```

```

[58]: # Calcul de l'âge
df_final['age'] = 2025 - df_final['birth']

```

```

[59]: # Définition des tranches d'âge personnalisées
bins = [0, 24, 34, 44, 54, 64, 74, 150]
labels = ['<25', '25-34', '35-44', '45-54', '55-64', '65-74', '75+']
df_final['tranche_age'] = pd.cut(df_final['age'], bins=bins, labels=labels,
    ↪right=False)

```

```

[60]: clients_par_age = df_final.groupby('tranche_age')['client_id'].nunique().
    ↪reset_index()
clients_par_age.columns = ['tranche_age', 'nb_clients']

fig = px.bar(
    clients_par_age,
    x='tranche_age',
    y='nb_clients',
    title='Répartition du nombre de clients par tranches d'âge',
    labels={'tranche_age': 'Tranche d'âge', 'nb_clients': 'Nombre de clients'},
    text='nb_clients',
    width=800,
    height=500
)

fig.update_traces(textposition='outside')
fig.update_layout(
    template='plotly_white',
    xaxis_title='Tranche d'âge',
    yaxis_title='Nombre de clients'
)

fig.show()

```

```

[61]: super_clients = ['c_1609', 'c_4958', 'c_6714', 'c_3454']

# Construire la table des clients uniques avec un âge valide (hors super_
    ↪clients)
clients_unique = (
    df_final.loc[~df_final['client_id'].isin(super_clients), ['client_id',
    ↪'age']]
    .dropna(subset=['age'])

```

```

        .drop_duplicates(subset='client_id')
    )

    # Box plot de la répartition des âges
    fig = px.box(
        clients_unique,
        y='age',
        title="Répartition des âges (box plot)",
        width=400,
        height=500
    )
    fig.update_layout(template='plotly_white', yaxis_title="Âge",
        ↪xaxis_visible=False, xaxis_showticklabels=False)
    fig.show()

```

```

[62]: # Table clients uniques avec âge + genre
clients_unique = (
    df_final.loc[~df_final['client_id'].isin(super_clients), ['client_id',
    ↪'age', 'sex']]
    .dropna(subset=['age', 'sex'])
    .drop_duplicates(subset='client_id')
)

# (optionnel) libellés propres
clients_unique['genre'] = clients_unique['sex'].map({'f': 'Femme', 'm':
    ↪'Homme'}).fillna(clients_unique['sex'])

# Box plot des âges par genre
fig = px.box(
    clients_unique,
    x='genre', # ou 'sex' si tu ne crées pas 'genre'
    y='age',
    color='genre',
    title="Répartition des âges par genre (box plot)",
    width=700, height=500
)
fig.update_layout(template='plotly_white', xaxis_title="Genre",
    ↪yaxis_title="Âge")
fig.show()

```

```

[63]: # Calcul du CA par tranche d'âge
ca_par_tranche = df_final.groupby('tranche_age')['CA'].sum().reset_index()

# Graphique
fig = px.bar(
    ca_par_tranche,
    x='tranche_age',

```



```

y='CA',
title="Répartition du chiffre d'affaires par tranche d'âge",
labels={'tranche_age': 'Tranche d'âge', 'CA': 'Chiffre d'affaires (€)'},
text='CA',
width=900,
height=500
)

fig.update_traces(texttemplate='%{text:.0f} €', textposition='outside')
fig.update_layout(
    template='plotly_white',
    xaxis_title="Tranche d'âge",
    yaxis_title="Chiffre d'affaires (€)"
)

fig.show()

```

```

[64]: # Regrouper le CA par âge
ca_par_age = df_final.groupby('age', as_index=False)['CA'].sum()

# Graphique
fig = px.bar(
    ca_par_age,
    x='age',
    y='CA',
    title="Chiffre d'affaires total par âge",
    labels={'age': 'Âge', 'CA': 'Chiffre d'affaires (€)'},
    text='CA',
    width=1000,
    height=500
)

fig.update_traces(texttemplate='%{text:.0f} €', textposition='outside')
fig.update_layout(
    template='plotly_white',
    xaxis_title="Âge",
    yaxis_title="Chiffre d'affaires (€)"
)

fig.show()

```

```

[65]: df_final['month'] = df_final['date'].dt.to_period('M').dt.to_timestamp()

ca_sexe_mensuel = df_final.groupby(['month', 'sex'])['CA'].sum().reset_index()

fig = px.line(
    ca_sexe_mensuel,

```

```

    x='month',
    y='CA',
    color='sex',
    title='Évolution mensuelle du chiffre d'affaires par sexe',
    labels={'month': 'Mois', 'CA': 'Chiffre d'affaires (€)', 'sex': 'Sexe'},
    markers=True,
    width=900,
    height=500
)

fig.update_layout(template='plotly_white')
fig.show()

```

```

[66]: clients_sexe_mensuel = df_final.groupby(['month', 'sex'])['client_id'].
      ↪nunique().reset_index()
clients_sexe_mensuel.columns = ['month', 'sex', 'nb_clients']

fig = px.line(
    clients_sexe_mensuel,
    x='month',
    y='nb_clients',
    color='sex',
    title='Évolution mensuelle du nombre de clients par sexe',
    labels={'month': 'Mois', 'nb_clients': 'Nombre de clients', 'sex': 'Sexe'},
    markers=True,
    width=900,
    height=500
)

fig.update_layout(template='plotly_white')
fig.show()

```

```

[67]: panier_moyen = df_final.groupby(['sex', 'tranche_age']).agg({
      'CA': 'sum',
      'session_id': 'nunique'
    }).reset_index()

panier_moyen['panier_moyen'] = panier_moyen['CA'] / panier_moyen['session_id']

fig = px.bar(
    panier_moyen,
    x='tranche_age',
    y='panier_moyen',
    color='sex',
    barmode='group',
    title='Panier moyen par sexe et tranche d'âge',

```

```

        labels={'tranche_age': 'Tranche d'âge', 'panier_moyen': 'Panier moyen (€)',
        ↪ 'sex': 'Sexe'},
        width=900,
        height=500
    )

fig.update_layout(template='plotly_white')
fig.show()

```

```

[68]: transactions_client = df_final.groupby(['client_id', 'sex',
        ↪ 'age'])['session_id'].nunique().reset_index()
transactions_moyennes = transactions_client.groupby(['sex',
        ↪ 'age'])['session_id'].mean().reset_index()
transactions_moyennes.columns = ['sex', 'age', 'transactions_moyennes']

fig = px.line(
    transactions_moyennes,
    x='age',
    y='transactions_moyennes',
    color='sex',
    title='Nombre moyen de transactions par client selon l'âge et le sexe',
    labels={'age': 'Âge', 'transactions_moyennes': 'Nombre moyen de
        ↪ transactions'},
    width=900,
    height=500
)

fig.update_layout(template='plotly_white')
fig.show()

```

```

[69]: freq_achat = df_final.groupby(['client_id', 'sex', 'tranche_age'])['date'].
        ↪ nunique().reset_index()
freq_moyenne = freq_achat.groupby(['sex', 'tranche_age'])['date'].mean().
        ↪ reset_index()
freq_moyenne.columns = ['sex', 'tranche_age', 'frequence_moy']

fig = px.bar(
    freq_moyenne,
    x='tranche_age',
    y='frequence_moy',
    color='sex',
    barmode='group',
    title='Fréquence d'achat moyenne par sexe et tranche d'âge',
    labels={'frequence_moy': 'Nombre moyen de jours distincts d'achat'},
    width=900,
    height=500
)

```

```
fig.update_layout(template='plotly_white')
fig.show()
```

```
[70]: # 1) CA cumulé par client (hors super clients)
super_clients = ['c_1609', 'c_4958', 'c_6714', 'c_3454']
ca_total_par_client = (
    df_final.loc[~df_final['client_id'].isin(super_clients), ['client_id', 'CA']]
    .groupby('client_id', as_index=False)['CA'].sum()
)

# 2) Créer des classes de CA
bins = [0, 100, 500, 1000, 5000, 10000, float('inf')]
labels = ['<100€', '100-500€', '500-1000€', '1k-5k€', '5k-10k€', '10k€+']
ca_total_par_client['classe_CA'] = pd.cut(
    ca_total_par_client['CA'], bins=bins, labels=labels, include_lowest=True,
    right=False
)

# 3) Histogramme (barres) du nombre de clients par classe de CA
fig = px.histogram(
    ca_total_par_client,
    x='classe_CA',
    title="Répartition des clients selon classes de chiffre d'affaires cumulé",
    labels={'classe_CA': 'Classe de CA (€)', 'count': 'Nombre de clients'},
    width=800, height=500, text_auto=True
)
fig.update_layout(template='plotly_white')
fig.update_xaxes(categoryorder='array', categoryarray=labels)
fig.show()
```

```
[71]: nb_categs = df_final.groupby('client_id')['categ'].nunique().reset_index()
nb_categs.columns = ['client_id', 'nb_categs']

fig = px.histogram(
    nb_categs,
    x='nb_categs',
    nbins=10,
    title="Nombre de catégories différentes achetées par client",
    labels={'nb_categs': 'Nombre de catégories'},
    width=800,
    height=500
)

fig.update_layout(template='plotly_white')
fig.show()
```

```
[72]: produits_clients = df_final.groupby('client_id')['id_prod'].nunique().
      ↪reset_index()
      produits_clients['type'] = produits_clients['id_prod'].apply(lambda x:
      ↪'Mono-produit' if x == 1 else 'Multi-produits')

      fig = px.pie(
          produits_clients,
          names='type',
          title='Répartition des clients mono-produit vs multi-produits',
          width=700,
          height=500
      )

      fig.update_traces(textinfo='percent+label')
      fig.update_layout(template='plotly_white')
      fig.show()

[73]: mois_clients = df_final.groupby('client_id')['month'].nunique().reset_index()
      mois_clients['type'] = mois_clients['month'].apply(lambda x: 'Fidèle' if x > 1
      ↪else 'Occasionnel')

      fig = px.pie(
          mois_clients,
          names='type',
          title='Répartition des clients fidèles vs occasionnels (mois différents)',
          width=700,
          height=500
      )

      fig.update_traces(textinfo='percent+label')
      fig.update_layout(template='plotly_white')
      fig.show()

[121]: import numpy as np
      import plotly.graph_objects as go

      # 1. CA cumulé par client
      ca_total_par_client = df_final.groupby('client_id')['CA'].sum().reset_index()

      # 2. Tri croissant par CA
      ca_sorted = ca_total_par_client.sort_values(by='CA').reset_index(drop=True)

      # 3. Calcul de la courbe de Lorenz
      ca_sorted['CA_cum'] = ca_sorted['CA'].cumsum()
      ca_sorted['CA_cum_percent'] = ca_sorted['CA_cum'] / ca_sorted['CA'].sum()
      ca_sorted['clients_percent'] = np.linspace(0, 1, len(ca_sorted))
```

```

# 4. Ajout du point (0,0)
x = np.insert(ca_sorted['clients_percent'].values, 0, 0)
y = np.insert(ca_sorted['CA_cum_percent'].values, 0, 0)

# 5. Tracé avec Plotly
fig = go.Figure()

fig.add_trace(go.Scatter(x=x, y=y, mode='lines', name='Courbe de Lorenz',
    ↪line=dict(color='royalblue'))))
fig.add_trace(go.Scatter(x=[0,1], y=[0,1], mode='lines', name='Égalité
    ↪parfaite', line=dict(dash='dash', color='gray'))))

fig.update_layout(
    title=" Courbe de Lorenz - Répartition du chiffre d'affaires entre les
    ↪clients",
    xaxis_title="Part cumulée des clients",
    yaxis_title="Part cumulée du chiffre d'affaires",
    width=700,
    height=800,
    template='plotly_white'
)

```

```

[120]: # Regrouper le CA par produit
df_lorenz_prod = df_final.groupby("id_prod", as_index=False)["CA"].sum()

# Trier les produits par CA croissant
df_lorenz_prod = df_lorenz_prod.sort_values("CA")

# Calcul des cumuls
df_lorenz_prod["cumulative_CA"] = df_lorenz_prod["CA"].cumsum()
df_lorenz_prod["cumulative_CA_percent"] = df_lorenz_prod["cumulative_CA"] /
    ↪df_lorenz_prod["CA"].sum()
df_lorenz_prod["cumulative_products_percent"] = np.arange(1,
    ↪len(df_lorenz_prod) + 1) / len(df_lorenz_prod)

# Tracer la courbe de Lorenz
fig = px.line(df_lorenz_prod,
    x="cumulative_products_percent",
    y="cumulative_CA_percent",
    title="Courbe de Lorenz - Répartition du chiffre d'affaires par
    ↪produit",
    labels={
        "cumulative_products_percent": "Proportion cumulée des
    ↪produits",
        "cumulative_CA_percent": "Proportion cumulée du chiffre
    ↪d'affaires"
    })

```

```

    },
    width=700,
    height=800)

# Ajouter la diagonale d'égalité parfaite
fig.add_scatter(x=[0, 1], y=[0, 1], mode="lines", name="Égalité parfaite",
    ↪line=dict(dash="dash"))

fig.show()

```

```

[76]: ca_par_client = df_final.groupby('client_id', as_index=False)['CA'].sum().
    ↪sort_values(by='CA', ascending=False)

fig = px.bar(
    ca_par_client.head(20),
    x='client_id',
    y='CA',
    title="Top 20 clients par chiffre d'affaires",
    labels={'client_id': 'Client', 'CA': 'Chiffre d'affaires (€)'},
    text='CA'
)

fig.update_traces(texttemplate='%{text:.0f} €', textposition='outside')
fig.update_layout(template="plotly_white")
fig.show()

```

```

[77]: super_clients = ['c_1609', 'c_4958', 'c_6714', 'c_3454']
df_clients_clean = df_final[~df_final['client_id'].isin(super_clients)]

```

```

[78]: # Agréger le CA par client
ca_clients = df_clients_clean.groupby('client_id', as_index=False)['CA'].sum()
ca_sorted = ca_clients.sort_values(by='CA')

# Calcul de la courbe de Lorenz
n = len(ca_sorted)
lorenz_y = np.cumsum(ca_sorted['CA'].values) / ca_sorted['CA'].sum()
lorenz_x = np.arange(1, n+1) / n

# Construction du DataFrame pour Plotly
lorenz_df = pd.DataFrame({
    'Part_cumulée_clients': lorenz_x,
    'Part_cumulée_CA': lorenz_y
})

# Tracer la courbe de Lorenz
fig = px.line(
    lorenz_df,

```

```

x='Part_cumulée_clients',
y='Part_cumulée_CA',
title="Courbe de Lorenz du chiffre d'affaires (hors super clients)",
labels={
    'Part_cumulée_clients': 'Part cumulée des clients',
    'Part_cumulée_CA': 'Part cumulée du chiffre d'affaires'
}
)

# Ajouter la ligne d'égalité parfaite
fig.add_scatter(x=[0,1], y=[0,1], mode='lines', name='Égalité parfaite',
               line=dict(dash='dash'))

fig.update_layout(template="plotly_white")
fig.show()

```

```

[79]: ca_par_age = df_final.groupby('age', as_index=False)['CA'].sum()

fig = px.bar(
    ca_par_age,
    x='age',
    y='CA',
    title="Chiffre d'affaires total par âge",
    labels={'age': 'Âge', 'CA': 'Chiffre d'affaires (€)'},
    text='CA',
    width=1000,
    height=500
)

fig.update_traces(texttemplate='%{text:.0f} €', textposition='outside')
fig.update_layout(
    template='plotly_white',
    xaxis_title="Âge",
    yaxis_title="Chiffre d'affaires (€)"
)

fig.show()

```

```

[80]: # Regrouper par âge et compter les clients uniques
clients_par_age = df_final.groupby('age')['client_id'].nunique().reset_index()
clients_par_age.columns = ['age', 'nb_clients']

# Graphique avec Plotly
fig = px.bar(
    clients_par_age,
    x='age',
    y='nb_clients',

```



```

        title="Nombre de clients par âge",
        labels={'age': 'Âge', 'nb_clients': 'Nombre de clients'},
        text='nb_clients',
        width=1000,
        height=500
    )

fig.update_traces(texttemplate='%{text}', textposition='outside')
fig.update_layout(
    template='plotly_white',
    xaxis_title="Âge",
    yaxis_title="Nombre de clients"
)

fig.show()

```

```

[81]: # Calcul du CA total et du nombre de clients par âge
panier_par_age = df_final.groupby('age').agg({
    'CA': 'sum',
    'client_id': 'nunique'
}).reset_index()

# Calcul du panier moyen (CA moyen par client)
panier_par_age['panier_moyen'] = panier_par_age['CA'] / \
    panier_par_age['client_id']

# Graphique
fig = px.bar(
    panier_par_age,
    x='age',
    y='panier_moyen',
    title="Panier moyen par âge (CA moyen par client)",
    labels={'age': 'Âge', 'panier_moyen': 'Panier moyen (€)'},
    text='panier_moyen',
    width=1000,
    height=500
)

fig.update_traces(texttemplate='%{text:.0f} €', textposition='outside')
fig.update_layout(
    template='plotly_white',
    xaxis_title="Âge",
    yaxis_title="Panier moyen (€)"
)

fig.show()

```

```
[82]: # Si jamais la colonne 'month' existe, on la supprime
if 'month' in df_final.columns:
    df_final.drop(columns=['month'], inplace=True)

# Convertir proprement les dates, même si la colonne est corrompue
df_final['date'] = pd.to_datetime(df_final['date'], errors='coerce')

# Créer la colonne 'month' sans utiliser .dt
df_final['month'] = df_final['date'].apply(lambda x: x.strftime('%Y-%m') if pd.
    ↪ notnull(x) else None)

# Supprimer les lignes sans mois (où date = NaT)
df_temp = df_final.dropna(subset=['month'])

# Regrouper CA par âge et par mois
ca_age_mois = df_temp.groupby(['age', 'month'], as_index=False)['CA'].sum()

# Tracer le graphique
fig = px.line(
    ca_age_mois,
    x='month',
    y='CA',
    color='age',
    title="Évolution du chiffre d'affaires par âge et par mois",
    labels={'month': 'Mois', 'CA': 'Chiffre d'affaires (€)', 'age': 'Âge'},
    width=1100,
    height=600
)

fig.update_layout(
    template='plotly_white',
    xaxis_title="Mois",
    yaxis_title="Chiffre d'affaires (€)",
    hovermode="x unified"
)

fig.show()
```

```
[83]: # Étape 1 : regrouper le CA total par âge
top_ages = df_temp.groupby('age')['CA'].sum().nlargest(10).index

# Étape 2 : filtrer le DataFrame sur ces âges
ca_top_age_mois = df_temp[df_temp['age'].isin(top_ages)].groupby(['age', 'month'],
    ↪ as_index=False)['CA'].sum()

# Étape 3 : tracer le graphique
fig = px.line(
```

```

    ca_top_age_mois,
    x='month',
    y='CA',
    color='age',
    title="Évolution du chiffre d'affaires par mois - Top 10 âges",
    labels={'month': 'Mois', 'CA': 'Chiffre d'affaires (€)', 'age': 'Âge'},
    width=1100,
    height=600
)

fig.update_layout(
    template='plotly_white',
    xaxis_title="Mois",
    yaxis_title="Chiffre d'affaires (€)",
    hovermode="x unified"
)

fig.show()

```

6 Correlations

6.1 Le lien entre le genre d'un client et les catégories des livres achetés

```

[84]: # 1) Tableau de contingence
df_final['genre'] = df_final['sex'].map({'f': 'Femme', 'm': 'Homme'}).
    ↪ fillna(df_final['sex'])
cont = pd.crosstab(df_final['genre'], df_final['categ'])

```

```

[85]: # 2) Test du Khi2 (stat + ddl + p-value)
from scipy.stats import chi2_contingency
chi2, p, dof, expected = chi2_contingency(cont)
print(f"Statistique Khi2 : {chi2:.2f} | ddl : {dof}")
print(f"p-value (notation scientifique) : {p:.3e}")

```

```

Statistique Khi2 : 154.75 | ddl : 2
p-value (notation scientifique) : 2.489e-34

```

```

[86]: # (option) p-value en décimal long
import decimal
decimal.getcontext().prec = 50
print("p-value (décimal) :", decimal.Decimal(p))

```

```

p-value (décimal) : 2.4885741867882007472738961481021403039474149264377073186330
114111446759828384374508043080410912839539605556637980043888092041015625E-34

```

```

[87]: # 3) Heatmap des effectifs (tableau de contingence coloré)
import plotly.express as px
fig = px.imshow(

```

```

    cont,
    text_auto=True,
    color_continuous_scale='Blues',
    labels=dict(x='Catégorie', y='Genre', color="Nombre d'achats"),
    title="Genre × Catégorie - effectifs"
)
fig.update_layout(template='plotly_white')
fig.show()

```

```

[88]: # 4) Heatmap des pourcentages par genre (lecture plus simple)
cont_pct = cont.div(cont.sum(axis=1), axis=0) * 100
fig = px.imshow(
    cont_pct.round(1),
    text_auto=True,
    color_continuous_scale='Blues',
    labels=dict(x='Catégorie', y='Genre', color="% dans le genre"),
    title="Genre × Catégorie - répartition (%) par genre"
)
fig.update_layout(template='plotly_white')
fig.show()

```

```

[89]: import numpy as np

# cont = pd.crosstab(df_final['genre'], df_final['categ'])
chi2, p, dof, expected = chi2_contingency(cont)
n = cont.to_numpy().sum()
r, c = cont.shape
v_cramer = np.sqrt(chi2 / (n * (min(r, c) - 1)))
print(f"V de Cramér : {v_cramer:.3f}") # ~0.1=faible, ~0.3=moyen, ~0.5=fort

```

V de Cramér : 0.015

```

[90]: resid_std = (cont - expected) / np.sqrt(expected)
import plotly.express as px
fig = px.imshow(
    resid_std, text_auto=".1f", color_continuous_scale="RdBu", origin="upper",
    labels=dict(x="Catégorie", y="Genre", color="Résidu std."),
    title="Genre × Catégorie - Résidus standardisés (écarts à l'indépendance)"
)
fig.update_layout(template="plotly_white")
fig.show()

```

6.2 le lien entre l'âge des clients et le montant total des achats

```

[91]: # --- Préparation : client-level + exclusion des super clients ---
super_clients = ['c_1609', 'c_4958', 'c_6714', 'c_3454']

X = (df_final.loc[~df_final['client_id'].isin(super_clients)])

```

```
.groupby('client_id', as_index=False)
.agg(age=('age', 'first'),
      CA=('CA', 'sum'))
.dropna())
```

[92]: # --- 1) Tests de normalité (Shapiro, sur un échantillon si très grand n) ---

```
from scipy.stats import shapiro, pearsonr, spearmanr
import numpy as np

samp = X.sample(n=min(5000, len(X)), random_state=42)

stat_age, p_age = shapiro(samp['age'])
stat_ca, p_ca = shapiro(samp['CA'])
print(f"Shapiro âge : stat={stat_age:.3f}, p={p_age:.3g}")
print(f"Shapiro CA : stat={stat_ca:.3f}, p={p_ca:.3g}")
```

Shapiro âge : stat=0.970, p=4.11e-31

Shapiro CA : stat=0.906, p=3.69e-48

[93]: # --- 2) Choix automatique du bon coefficient ---

```
if (p_age > 0.05) and (p_ca > 0.05):
    r, p = pearsonr(X['age'], X['CA'])
    methode = "Pearson (distributions ~ normales)"
else:
    r, p = spearmanr(X['age'], X['CA'])
    methode = "Spearman (au moins une distribution non normale)"

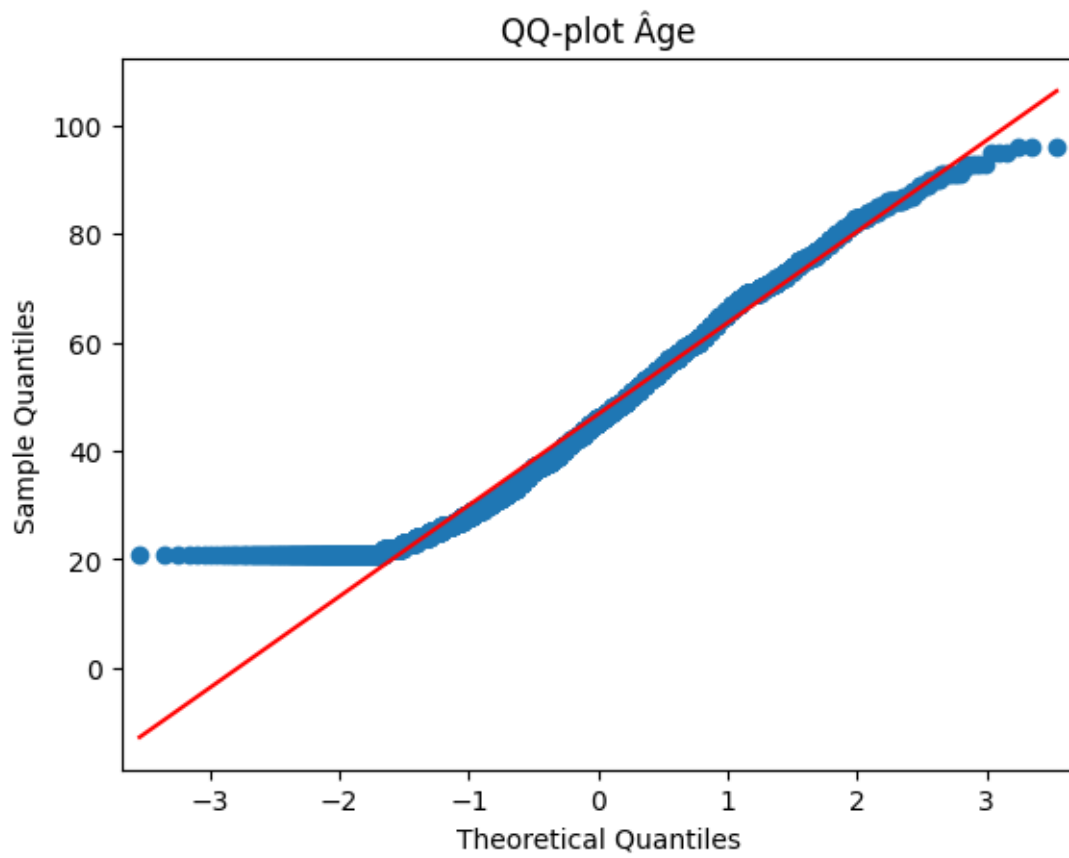
print(f"Méthode : {methode}")
print(f"Coefficient r = {r:.3f} | p-value = {p:.3g}")
```

Méthode : Spearman (au moins une distribution non normale)

Coefficient r = -0.184 | p-value = 1.41e-66

[94]: # --- 3) QQ-plots (visuel de normalité) ---

```
import statsmodels.api as sm
import matplotlib.pyplot as plt
sm.qqplot(samp['age'], line='s'); plt.title("QQ-plot Âge"); plt.show()
sm.qqplot(samp['CA'], line='s'); plt.title("QQ-plot CA total par client"); plt.
    show()
```





[95]: # --- 4) Scatter avec un point par âge (moyenne) + régression rouge ---

```
import plotly.express as px
age_mean = X.groupby('age', as_index=False)['CA'].mean()

fig = px.scatter(
    age_mean, x='age', y='CA',
    trendline='ols', trendline_color_override='red',
    title="Âge (point moyen par âge) vs CA moyen",
    labels={'age': 'Âge', 'CA': "CA moyen (€)"}
)
fig.update_traces(marker=dict(opacity=0.7, size=7))
fig.update_layout(template='plotly_white')
fig.show()
```

[96]: # --- 5) (option) Deux tendances séparées

```
def corr_segment(df, a, b):
    seg = df[(df['age']>=a) & (df['age']<=b)]
    # on reprend la même logique de choix Pearson/Spearman selon Shapiro dans
    ↪ le segment
```

```

ss = seg.sample(n=min(5000, len(seg)), random_state=42) if len(seg)>0 else_
↪seg
pa = shapiro(ss['age'])[1] if len(ss)>3 else 1.0
pc = shapiro(ss['CA'])[1] if len(ss)>3 else 1.0
if (pa>0.05) and (pc>0.05):
    rr, pp = pearsonr(seg['age'], seg['CA']); meth="Pearson"
else:
    rr, pp = spearmanr(seg['age'], seg['CA']); meth="Spearman"
print(f"[{a}-{b}] {meth} : r={rr:.3f} | p={pp:.3g}")

corr_segment(X, 21, 50)
corr_segment(X, 50, 95)

```

[21-50] Spearman : r=0.073 | p=1.45e-07
[50-95] Spearman : r=-0.163 | p=1.12e-22

```

[97]: # 1) 1 point par âge = CA moyen pour chaque âge
age_mean = (X.groupby('age', as_index=False)['CA'].mean()
            .sort_values('age'))

# 2) Segmenter les âges en 20-53 et 54-95
age_mean = age_mean.query('age >= 20 and age <= 95').copy()
age_mean['segment'] = pd.cut(
    age_mean['age'],
    bins=[20, 53, 95],          # 20-53 puis 54-95
    labels=['20-53 ans', '54-95 ans'],
    include_lowest=True, right=True
)

# 3) Scatter avec une régression linéaire par segment (rouge)
fig = px.scatter(
    age_mean, x='age', y='CA',
    color='segment',
    trendline='ols', trendline_color_override='red',
    labels={'age': 'Âge', 'CA': 'CA moyen (€)', 'segment': "Segment d'âge"},
    title="Âge (1 point par âge) vs CA moyen - segments 20-53 et 54-95"
)
fig.update_traces(marker=dict(size=8, opacity=0.8))
fig.update_layout(template='plotly_white')
fig.show()

```

6.3 Le lien entre l'âge des clients et la fréquence d'achat

```

[98]: # 0) Prépa des données : 1 ligne = 1 client, fréquence = nb de sessions uniques
super_clients = ['c_1609', 'c_4958', 'c_6714', 'c_3454']

```



```
base = df_final.loc[~df_final['client_id'].isin(super_clients), ['client_id', 'age', 'session_id']].dropna()
freq_client = (
    base.groupby('client_id', as_index=False)
        .agg(age=('age', 'first'), freq=('session_id', 'nunique'))
)
```

```
[99]: # 1) Tests de normalité (Shapiro) pour décider Pearson vs Spearman
from scipy.stats import shapiro, pearsonr, spearmanr
samp = freq_client.sample(n=min(5000, len(freq_client)), random_state=42)

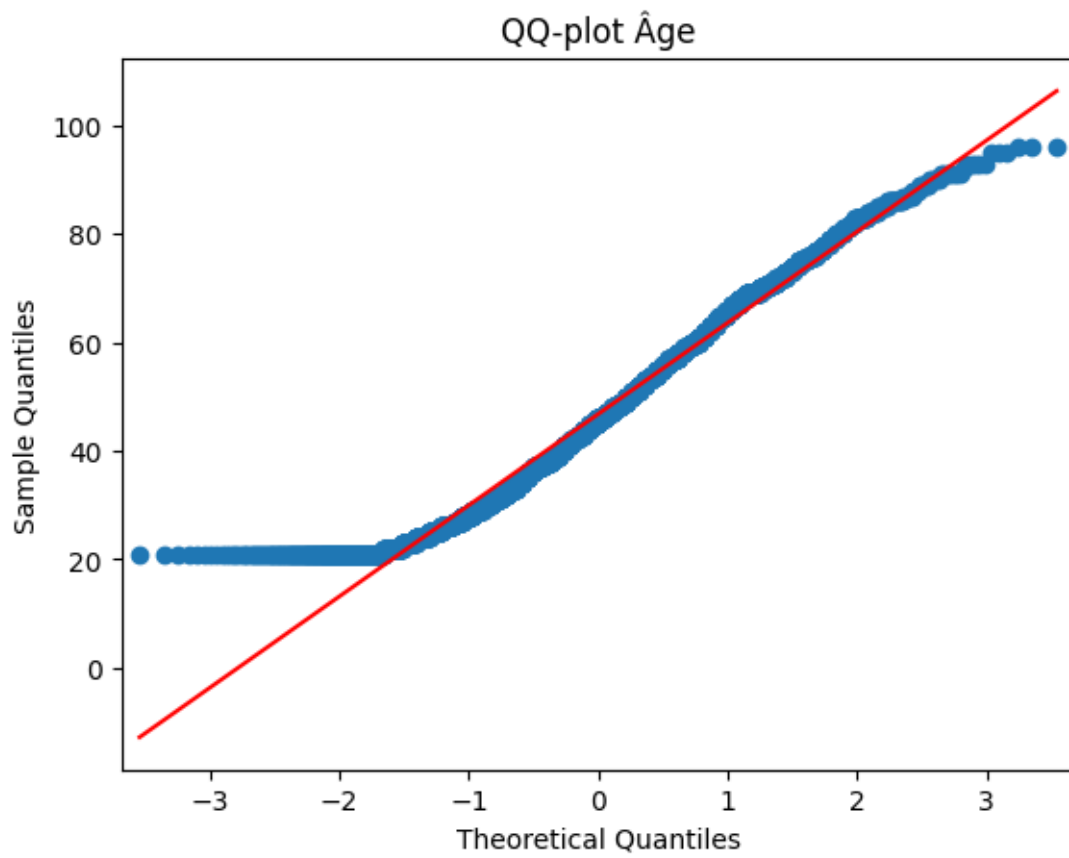
stat_age, p_age = shapiro(samp['age'])
stat_frq, p_frq = shapiro(samp['freq'])
print(f"Shapiro âge : stat={stat_age:.3f}, p={p_age:.3g}")
print(f"Shapiro fréquence : stat={stat_frq:.3f}, p={p_frq:.3g}")
```

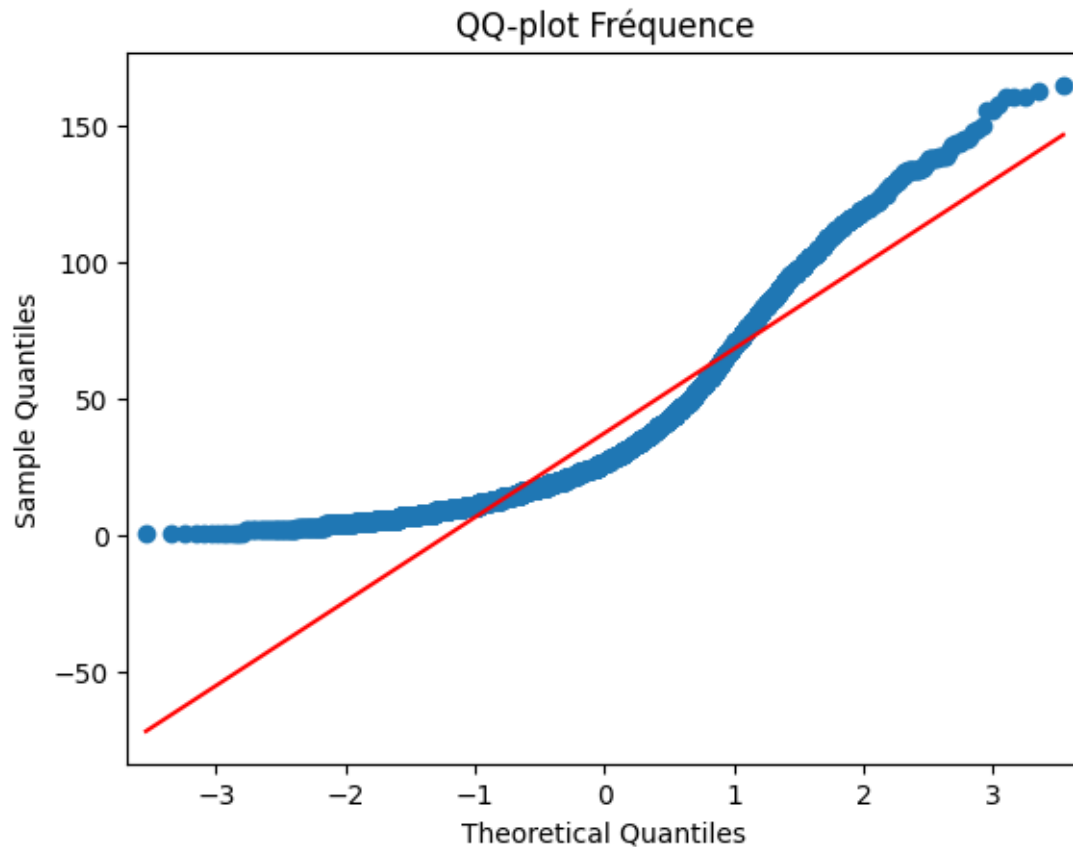
Shapiro âge : stat=0.970, p=4.11e-31
 Shapiro fréquence : stat=0.858, p=2.46e-55

```
[100]: # 2) Choix du coefficient selon normalité
if (p_age > 0.05) and (p_frq > 0.05):
    r, p = pearsonr(freq_client['age'], freq_client['freq'])
    methode = "Pearson (distributions ~ normales)"
else:
    r, p = spearmanr(freq_client['age'], freq_client['freq'])
    methode = "Spearman (au moins une non normale)"
print(f"Méthode : {methode}")
print(f"Coefficient r = {r:.3f} | p-value = {p:.3g}")
```

Méthode : Spearman (au moins une non normale)
 Coefficient r = 0.212 | p-value = 6.63e-88

```
[101]: # 3) QQ-plots (visuel normalité)
import statsmodels.api as sm
import matplotlib.pyplot as plt
sm.qqplot(samp['age'], line='s'); plt.title("QQ-plot Âge"); plt.show()
sm.qqplot(samp['freq'], line='s'); plt.title("QQ-plot Fréquence"); plt.show()
```





```
[102]: # 4) Scatter avec un seul point par âge (moyenne) + régression rouge
import plotly.express as px
age_mean_freq = freq_client.groupby('age', as_index=False)['freq'].mean()

fig = px.scatter(
    age_mean_freq, x='age', y='freq',
    trendline='ols', trendline_color_override='red',
    title="Âge (point moyen par âge) vs fréquence d'achat",
    labels={'age': 'Âge', 'freq': 'Fréquence (moyenne de sessions uniques)'}
)
fig.update_traces(marker=dict(size=7, opacity=0.8))
fig.update_layout(template='plotly_white')
fig.show()
```

```
[103]: # 5) (option) Corrélacion par segments d'âge (21-50) et (50-95)
from scipy.stats import shapiro
def corr_segment_freq(df, a, b):
    seg = df[(df['age']>=a) & (df['age']<=b)]
    if len(seg) < 4:
        print(f"[{a}-{b}] échantillon trop petit"); return
```

```

ss = seg.sample(n=min(5000, len(seg)), random_state=42)
pa = shapiro(ss['age'])[1]; pf = shapiro(ss['freq'])[1]
if (pa > 0.05) and (pf > 0.05):
    rr, pp = pearsonr(seg['age'], seg['freq']); meth = "Pearson"
else:
    rr, pp = spearmanr(seg['age'], seg['freq']); meth = "Spearman"
print(f"[{a}-{b}] {meth} : r={rr:.3f} | p={pp:.3g}")

corr_segment_freq(freq_client, 21, 50)
corr_segment_freq(freq_client, 50, 95)

```

[21-50] Spearman : r=0.429 | p=2.03e-231

[50-95] Spearman : r=-0.091 | p=5.15e-08

```

[104]: # Agréger : un seul point par âge
age_mean_freq = (freq_client.groupby('age', as_index=False)['freq'].mean()
                .sort_values('age'))

# Découper en deux segments
age_mean_freq = age_mean_freq.query('age >= 20 and age <= 95').copy()
age_mean_freq['segment'] = pd.cut(
    age_mean_freq['age'],
    bins=[20, 53, 95],
    labels=['20-53 ans', '54-95 ans'],
    include_lowest=True, right=True
)

# Scatter + régressions linéaires rouges par segment
fig = px.scatter(
    age_mean_freq, x='age', y='freq', color='segment',
    trendline='ols', trendline_color_override='red',
    labels={'age': 'Âge', 'freq': 'Fréquence (moyenne de sessions uniques)',
    ↪ 'segment': "Segment d'âge"},
    title="Âge (1 point par âge) vs fréquence d'achat - segments 20-53 et 54-95"
)
fig.update_traces(marker=dict(size=8, opacity=0.85))
fig.update_layout(template='plotly_white')
fig.show()

```

6.4 Le lien entre l'âge des clients et la taille du panier moyen

```

[127]: # 0) Préparation (hors super clients)
super_clients = ['c_1609', 'c_4958', 'c_6714', 'c_3454']
base = df_final.loc[~df_final['client_id'].isin(super_clients),
    ↪ ['client_id', 'age', 'session_id', 'CA']].dropna()

# 1) Panier moyen par client = moyenne du CA par session (commande)

```

```
orders = base.groupby(['client_id', 'session_id'], as_index=False)['CA'].sum()
panier_client = (orders.groupby('client_id', as_index=False)['CA'].mean()
                  .rename(columns={'CA': 'panier_moyen'})
                  .merge(base[['client_id', 'age']].drop_duplicates(),
                        on='client_id', how='left'))
```

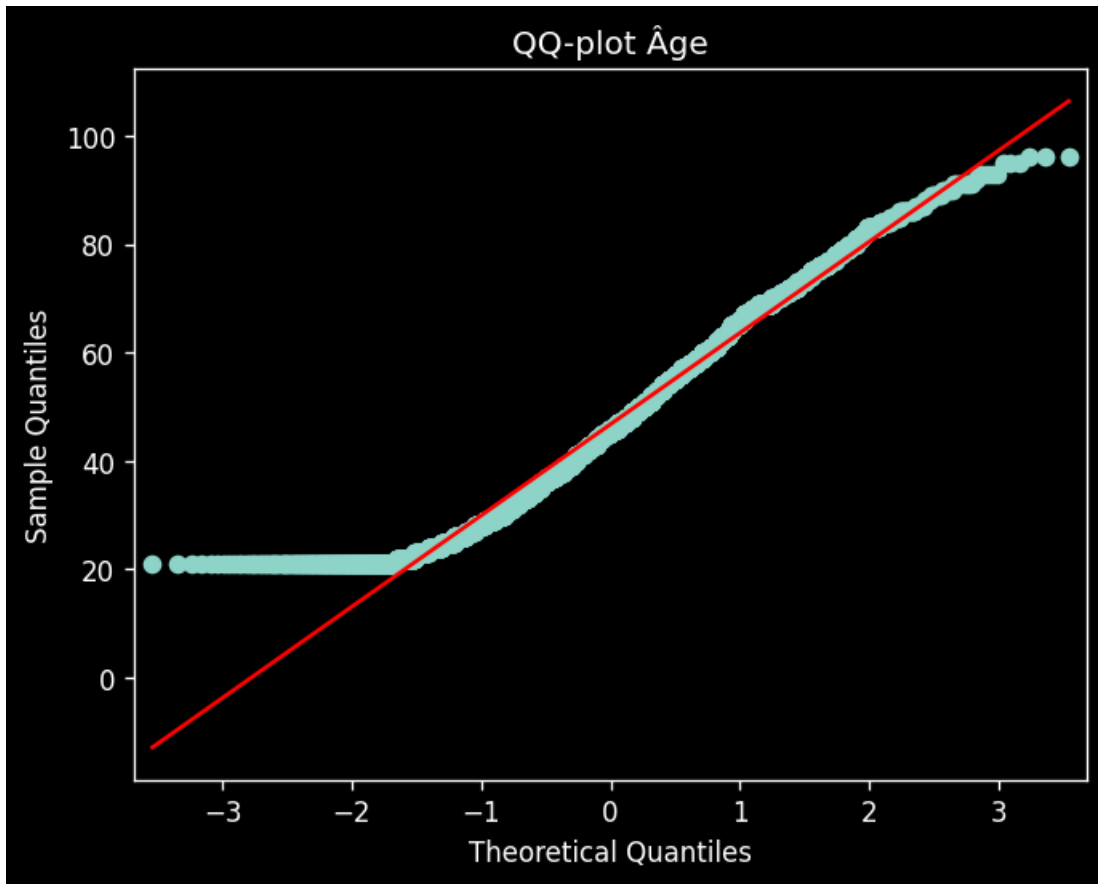
```
[128]: # 2) Tests de normalité (Shapiro) pour décider Pearson vs Spearman
from scipy.stats import shapiro, pearsonr, spearmanr
samp = panier_client.sample(n=min(5000, len(panier_client)), random_state=42)
stat_age, p_age = shapiro(samp['age'])
stat_pm, p_pm = shapiro(samp['panier_moyen'])
print(f"Shapiro âge : stat={stat_age:.3f}, p={p_age:.3g}")
print(f"Shapiro panier_moyen : stat={stat_pm:.3f}, p={p_pm:.3g}")
```

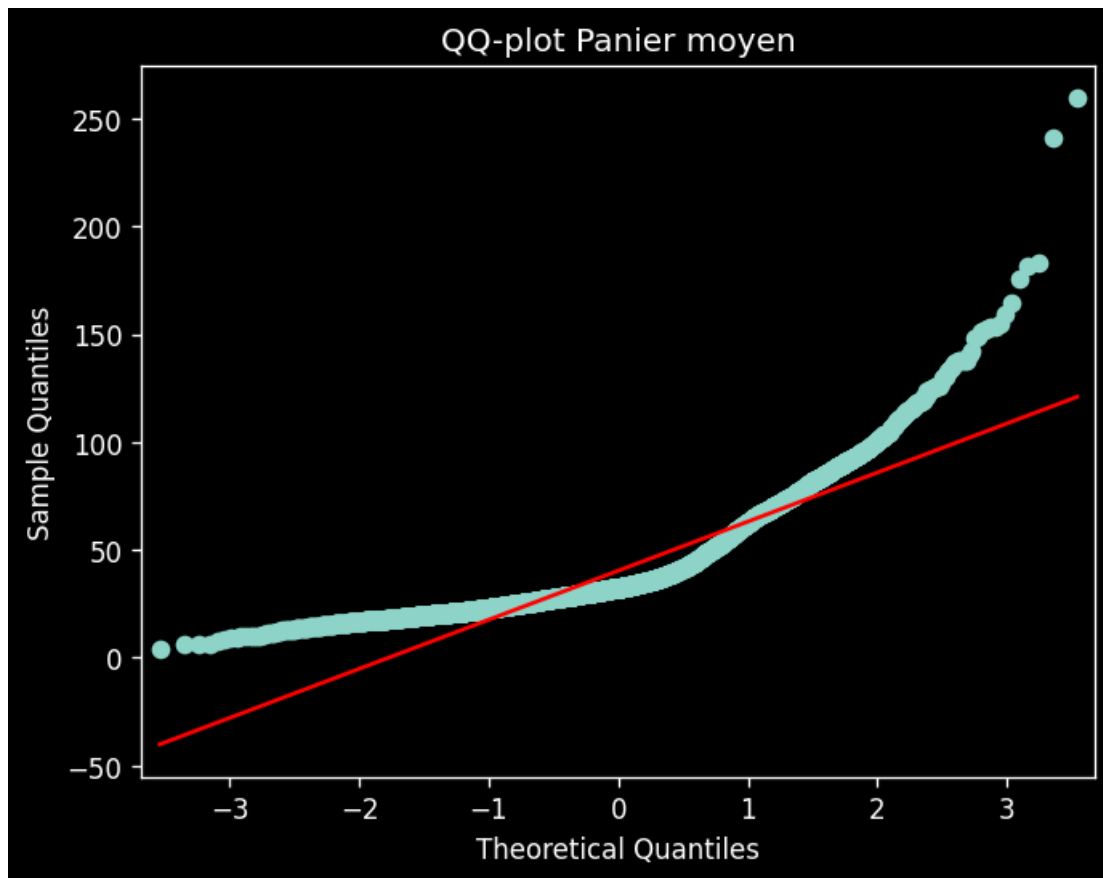
Shapiro âge : stat=0.970, p=4.11e-31
Shapiro panier_moyen : stat=0.812, p=2.21e-60

```
[129]: # 3) Coefficient de corrélation selon normalité
if (p_age > 0.05) and (p_pm > 0.05):
    r, p = pearsonr(panier_client['age'], panier_client['panier_moyen'])
    methode = "Pearson"
else:
    r, p = spearmanr(panier_client['age'], panier_client['panier_moyen'])
    methode = "Spearman"
print(f"Méthode : {methode} | r = {r:.3f} | p-value = {p:.3g}")
```

Méthode : Spearman | r = -0.701 | p-value = 0

```
[130]: # 4) QQ-plots (visuel)
import statsmodels.api as sm
import matplotlib.pyplot as plt
sm.qqplot(samp['age'], line='s'); plt.title("QQ-plot Âge"); plt.show()
sm.qqplot(samp['panier_moyen'], line='s'); plt.title("QQ-plot Panier moyen");
plt.show()
```





```
[131]: # 5) Scatter « un point par âge » + régression rouge
import plotly.express as px
pm_age = panier_client.groupby('age', as_index=False)['panier_moyen'].mean()
fig = px.scatter(pm_age, x='age', y='panier_moyen',
                 trendline='ols', trendline_color_override='red',
                 title="Âge (point moyen par âge) vs Panier moyen",
                 labels={'age': 'Âge', 'panier_moyen': 'Panier moyen (€)'},
                 width=900, height=500)
fig.update_traces(marker=dict(size=7, opacity=0.8))
fig.update_layout(template='plotly_white')
fig.show()
```

```
[132]: # 6) (option) Corrélations par segments d'âge (21-50) et (50-95)
def corr_segment_pm(df, a, b):
    seg = df[(df['age']>=a) & (df['age']<=b)]
    if len(seg) < 4:
        print(f"[{a}-{b}] échantillon trop petit"); return
    ss = seg.sample(n=min(5000, len(seg)), random_state=42)
    pa = shapiro(ss['age'])[1]; pp = shapiro(ss['panier_moyen'])[1]
    if (pa > 0.05) and (pp > 0.05):
```

```

    rr, pv = pearsonr(seg['age'], seg['panier_moyen']); meth = "Pearson"
else:
    rr, pv = spearmanr(seg['age'], seg['panier_moyen']); meth = "Spearman"
print(f"[{a}-{b}] {meth} : r={rr:.3f} | p-value={pv:.3g}")

```

```

corr_segment_pm(panier_client, 21, 50)
corr_segment_pm(panier_client, 50, 95)

```

```

[21-50] Spearman : r=-0.674 | p-value=0
[50-95] Spearman : r=-0.215 | p-value=1.13e-38

```

```

[133]: # 1) Corrélation client-level par segments (corr_segment_pm existant)
corr_segment_pm(panier_client, 20, 53)
corr_segment_pm(panier_client, 54, 95)

# 2) SCATTER avec un seul point par âge (panier moyen moyen) + régressions
↳ rouges
age_pm = (panier_client.groupby('age', as_index=False)['panier_moyen'].mean()
          .query('age >= 20 and age <= 95')
          .sort_values('age'))
age_pm['segment'] = pd.cut(
    age_pm['age'],
    bins=[20, 53, 95],
    labels=['20-53 ans', '54-95 ans'],
    include_lowest=True, right=True
)

fig = px.scatter(
    age_pm, x='age', y='panier_moyen', color='segment',
    trendline='ols', trendline_color_override='red',
    labels={'age': 'Âge', 'panier_moyen': 'Panier moyen (€)', 'segment': "Segment",
    ↳ d'âge"},
    title="Âge (1 point par âge) vs Panier moyen - segments 20-53 et 54-95"
)
fig.update_traces(marker=dict(size=8, opacity=0.85))
fig.update_layout(template='plotly_white')
fig.show()

# 3) (option) Corrélation sur les points moyens par âge, par segment
from scipy.stats import shapiro, pearsonr, spearmanr

def corr_segment_pm_mean(df_age_pm, a, b):
    seg = df_age_pm[(df_age_pm['age'] >= a) & (df_age_pm['age'] <= b)]
    if len(seg) < 4:
        print(f"[{a}-{b}] échantillon trop petit"); return
    pa = shapiro(seg['age'])[1]
    pp = shapiro(seg['panier_moyen'])[1]

```



```

if (pa > 0.05) and (pp > 0.05):
    r, p = pearsonr(seg['age'], seg['panier_moyen']); m = "Pearson"
else:
    r, p = spearmanr(seg['age'], seg['panier_moyen']); m = "Spearman"
print(f"[{a}]-[{b}] {m} (points moyens/âge) : r={r:.3f} | p-value={p:.3g}")

corr_segment_pm_mean(age_pm, 20, 53)
corr_segment_pm_mean(age_pm, 54, 95)

```

```

[20-53] Spearman : r=-0.653 | p-value=0
[54-95] Spearman : r=0.030 | p-value=0.108

[20-53] Spearman (points moyens/âge) : r=-0.653 | p-value=3.79e-05
[54-95] Pearson (points moyens/âge) : r=0.193 | p-value=0.22

```

6.5 Le lien entre l'âge des clients et la catégorie des livres achetés

```

[112]: # 0) Préparation : client-level, hors super clients, colonnes utiles
super_clients = ['c_1609', 'c_4958', 'c_6714', 'c_3454']
base = (
    df_final.loc[~df_final['client_id'].isin(super_clients),
    ↪ ['client_id', 'age', 'categ', 'CA']]
    .dropna(subset=['age', 'categ'])
)

```

```

[113]: # 1) Catégorie principale par client = celle où il dépense le plus (évite de
    ↪ dupliquer un client dans plusieurs groupes)
df_cc = base.groupby(['client_id', 'categ'], as_index=False)['CA'].sum()
dominant = df_cc.loc[df_cc.groupby('client_id')['CA'].idxmax()] \
    .merge(base[['client_id', 'age']].drop_duplicates(),
    ↪ on='client_id', how='left')

```

```

[114]: # 2) Tests de normalité (Shapiro) par catégorie
from scipy.stats import shapiro, f_oneway, kruskal

pvals = {}
for cat, grp in dominant.groupby('categ'):
    s = grp['age'].dropna()
    # Shapiro sur un échantillon si groupe très grand
    ss = s.sample(n=min(5000, len(s)), random_state=42) if len(s) > 0 else s
    stat, p = shapiro(ss)
    pvals[cat] = p
    print(f"Shapiro âge | catégorie {cat} : p={p:.3g} (n={len(s)})")

all_normal = all(p > 0.05 for p in pvals.values())

```

```

Shapiro âge | catégorie 0 : p=1.22e-40 (n=3072)
Shapiro âge | catégorie 1 : p=2.01e-28 (n=3509)
Shapiro âge | catégorie 2 : p=1.87e-53 (n=2015)

```

```
[115]: # 3) Choix du test et calcul
groups = [grp['age'].values for _, grp in dominant.groupby('categ') if len(grp) > 1]

if all_normal and len(groups) >= 2:
    stat, p = f_oneway(*groups) # ANOVA à 1 facteur
    methode = "ANOVA (normalité OK par groupe)"
else:
    stat, p = kruskal(*groups) # Non paramétrique sinon
    methode = "Kruskal-Wallis (au moins un groupe non normal)"

print(f"Méthode : {methode}")
print(f"Stat = {stat:.3f} | p-value = {p:.3g}")
```

Méthode : Kruskal-Wallis (au moins un groupe non normal)
Stat = 5056.048 | p-value = 0

```
[116]: # 4) Visuel : distribution des âges par catégorie (box plot)
fig = px.box(
    dominant, x='categ', y='age', color='categ',
    title="Âge des clients par catégorie (catégorie principale par client)",
    labels={'categ': 'Catégorie', 'age': 'Âge'},
    width=900, height=500
)
fig.update_layout(template='plotly_white', showlegend=False)
fig.show()
```